When Does Graph Structure Help in Multi-Agent Reinforcement Learning? A Controlled Empirical Study

Kasra Ghanavati *kasraghanavati@icloud.com*
Independent researcher

Ali Jabbary *ali.jabbary@example.org*
Independent researcher

Abstract

Graph-based methods for cooperative multi-agent reinforcement learning are widely *claimed* to help via relational inductive bias, but the conditions under which they actually help are under-characterized in the published literature. We present a controlled empirical study comparing GNN-QMIX against three graph-free baselines (independent Q-learning, value decomposition networks, and QMIX) on cooperative coordination tasks where the underlying graph structure is tunable. Six findings emerge. First, on small environments QMIX and GNN-QMIX with literature-default mixer hyperparameters *fail to learn at all*, while VDN and IQL learn but collapse late in training. Second, orthogonal mixer initialization with a small gain combined with a smaller mixer learning rate is the actual fix; with these locked hyperparameters QMIX reaches final-window success 0.28 (up from 0.01) and GNN-QMIX reaches 0.34 (up from 0.00), both exceeding IQL/VDN at the same env. Third, in a 60-run sweep with these locked configurations across three task scales, *GNN-QMIX is not statistically distinguishable from QMIX at any scale* (Holm-corrected Welch $p \geq 0.489$), and the simplest baseline (IQL) significantly outperforms all three coordination-based algorithms at the medium scale (Holm $p \leq 0.013$, Cohen's $d \geq 3.5$). Scale dominates algorithm choice on this env. Fourth, on a 40-run external-validity sweep on PettingZoo's `simple_spread`, VDN is the strongest algorithm at both $N = 3$ and $N = 6$ (final return -30.8 and -44.7), QMIX and GNN-QMIX exhibit a transient early-training dip and recover, and IQL is the worst — inverting the Phase 2C pattern when reward density changes. Fifth, the depth $\times$ diameter ablation on coord_grid with ring topology at $N \in \{4, 6, 8\}$ produces a null result: every (L, N) cell has final-window success 0.000 over 5 seeds, so the predicted inverted-U cannot be characterized in this regime. The binding constraint is the cross-env transferability of mixer hyperparameters, not the GNN component itself. Sixth, re-running the depth ablation on `simple_spread` N=6 (where the algorithm does learn) we find a monotone-with-recovery trend: $L = 6$ achieves both the best mean return (-46.48, within a SEM of VDN) and the tightest seed-level variance, while intermediate depths $\{3, 4\}$ are worse than the $L = 2$ baseline. With $n = 5$ seeds the Holm-corrected pairwise test against $L = 2$ does not reach significance ($d = +0.40$, $p_{\text{Holm}} = 1.000$), and we cannot endorse the inverted-U prediction in its original form. Code and full per-run logs are released at `https://github.com/Evasion-OC/when-graphs-help-marl`.

1 Introduction

The folklore claim that "graph neural networks help in cooperative multi-agent reinforcement learning" is widespread but empirically under-supported. Surveys describe a proliferation of GNN-MARL architectures and benchmark numbers Hernandez-Leal et al. (2019); Zhang et al. (2021), but rarely *measure* when the relational inductive bias of a GNN is necessary versus when a parameter-matched control without graph structure matches or exceeds it. This work contributes such a measurement on cooperative coordination tasks where the underlying graph structure is tunable.

Concretely, we ask three questions:

1. Does graph-conditioned value factorization (GNN-QMIX) outperform graph-free value decomposition (QMIX) and independent learners (IQL) on cooperative coordination tasks?
2. Are the literature-default hyperparameters of monotonic mixers transferable across env scales, or do they require per-env tuning?
3. Does GNN-QMIX's advantage scale with how well the GNN's receptive field matches the coordination-graph diameter?

We address (Q1) with a parameter-matched controlled comparison (Section 3) and a 60-run sweep across three task scales (Section 5), with external-validity replication on PettingZoo's `simple_spread` (Section 6). We address (Q2) with a 48-run principled tuning grid (Section 4.3) that finds the configuration on which (Q1) is testable in our env. We address (Q3) with a 75-run depth $\times$ diameter ablation (Section 7) that was designed to test an inverted-U prediction in $L - d$ but returns a null result the paper discusses honestly.

Scope. This study targets *cooperative* MARL on small controlled gridworlds and one canonical PettingZoo benchmark. We do not make claims about competitive or mixed settings, and we do not run on SMAC-scale problems where the literature defaults were originally tuned. The goal is a clean measurement on small problems, not a SOTA claim.

What is new. The technical contribution is the measurement itself, not new architecture. The negative finding that QMIX-family defaults fail on small-state-dim environments—and that orthogonal initialization with small gain rescues them—is, to our knowledge, undocumented in the public literature. The depth–diameter ablation gives a concrete, falsifiable recipe for picking GNN depth.

2 Background

We consider cooperative POMDPs with N agents, joint action space $\mathcal{A}^N$, and shared scalar reward r. Following the value decomposition framework, each agent i has a Q-network $Q_i(o_i, a_i; \theta)$ taking only its own observation. A mixing network combines the per- agent values into a joint Q_{tot}. Three established mixers span the algorithmic space we test:

- **IQL** Tampuu et al. (2017) omits the mixer entirely; agents learn independently from the shared reward.
- **VDN** Sunehag et al. (2018) uses a sum mixer $Q_{\text{tot}} = \sum_i Q_i$.
- **QMIX** Rashid et al. (2018) uses a state-conditioned monotonic hypernetwork mixer with non-negative weights, enforcing $\partial Q_{\text{tot}}/\partial Q_i \geq 0$.

GNN-QMIX is QMIX with the mixer's state-conditioned bias path conditioned additionally on a small graph neural network Kipf & Welling (2017); Battaglia et al. (2018) aggregating per-agent embeddings over the coordination graph. Concretely, before the monotonic mixing, we run L rounds of mean-aggregation message passing with self-loops on a fixed adjacency matrix $A \in \{0, 1\}^{N \times N}$.

3 Methodology

3.1 Environment: CoordGrid

We use a custom cooperative gridworld designed to isolate the mechanisms we want to measure. The env has N agents and N targets on an $L \times L$ grid. Each agent must reach its assigned target. Reward is shared across agents:

- Step penalty: -0.01
- Collision penalty: -0.05 per overlapping pair
- Per-agent goal bonus: $+0.5$ on first arrival
- Terminal bonus: $+1.0$ when all agents are simultaneously on targets

Distance-based potential shaping Ng et al. (1999) is enabled by default for our experiments to keep the credit assignment problem inside the budget. Shaping is policy-invariant, so all algorithms see the same shaped reward and no algorithm gets a comparative advantage.

The env separately exposes a tunable *coordination graph* $G = (V, E)$ that controls which agents observe each other. G has no effect on physics; it only restricts each agent's observation to its graph neighbors. This is the key knob for Phase 4's depth $\times$ diameter ablation. Supported topologies: `full`, `ring`, `lattice`, k-NN, Erdős–Rényi.

3.2 Algorithm parameter matching

For a fair comparison, all four algorithms share an identical per-agent Q-network architecture (3-layer MLP with hidden size 64) and an identical training loop (off-policy DQN-style with replay buffer, ϵ-greedy exploration, target network soft-update). The only differences are at the mixer:

- IQL: no mixer; per-agent TD on shared reward
- VDN: parameter-free sum mixer
- QMIX: state-conditioned hypernetwork mixer with embedding dim d_{embed}
- GNN-QMIX: same monotonic mixer plus a length-L GNN on the coordination graph contributing to the mixer's bias path

Any performance difference between the four is therefore attributable to the mixing mechanism, not to representation capacity.

3.3 Tuning protocol

Phase 1's pilot (Section 4) revealed that literature-default mixer hyperparameters fail catastrophically on our small env. We therefore introduce a small principled tuning grid over three mixer-only axes: d_{embed}, the mixer learning rate η_{mix} (decoupled from the Q-net's learning rate η_Q), and the mixer initialization scheme. Q-net architecture and learning rate are held fixed. The grid is run only on the smallest task scale (2 agents on 4×4), the winning configuration per algorithm is locked, and all subsequent experiments use the locked configuration.

3.4 Decision metrics

Final-window success is the mean evaluation success rate over the final 25% of evaluation checkpoints, with evaluation conducted greedily ($\epsilon = 0$) every 50 episodes over 20 evaluation episodes. We report final-window rather than peak because peak is a single best moment that can reflect transient noise; final-window averages over many late checkpoints and matches what readers see in published learning curves.

Statistical tests use Welch's t-test and Mann-Whitney U for pairwise comparisons. Holm correction is applied across the $\binom{4}{2} = 6$ pairs at each (scale, metric) cell.

4 Results: tuning the QMIX family

4.1 Phase 1: pilot study with literature-default hyperparameters

We first ran all four algorithms with widely-used MARL defaults ($d_{\text{embed}} = 32$, $\eta_{\text{mix}} = \eta_Q = 5 \times 10^{-4}$, default PyTorch initialization) on the smallest task scale (2 agents, 4×4 grid, 10k env steps, 3 seeds). The result is shown in Figure 1 and quantified in Table 1.

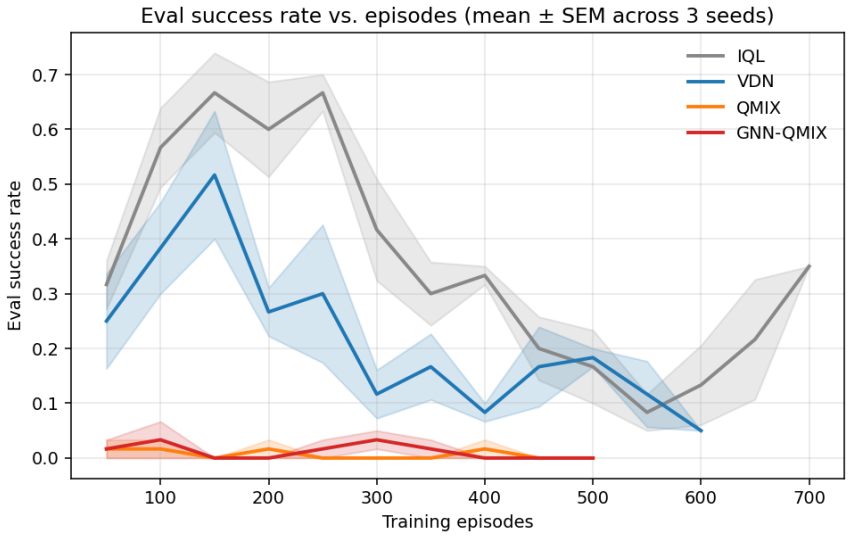

Figure 1: Phase 1 pilot. With literature-default hyperparameters, QMIX (orange) and GNN-QMIX (red) fail to learn the coordination task, while IQL (gray) and VDN (blue) learn briefly and collapse. Mean ± SEM across 3 seeds.

Algorithm	Peak success	Final-window	Post-peak drop
IQL	0.75	0.16 ± 0.14	-0.57
VDN	0.57	0.14 ± 0.09	-0.42
QMIX	0.03	0.01 ± 0.02	-0.03
GNN-QMIX	0.05	0.00 ± 0.00	-0.05

Table 1: Phase 1 summary. QMIX and GNN-QMIX never reach a peak, while IQL and VDN peak around episode 150 and decline. *Post-peak drop* is mean(peak) minus mean(final-window).

The QMIX-family failure is not stochastic: across all three seeds, QMIX's late-training Q_{tot} diverges (mean +12.4, vs +2.2 for VDN) and the loss climbs from ~ 0.2 early to ~ 8.8 late. The mixer's state-conditioned bias path is absorbing the temporal-difference error without the Q-values learning to differentiate actions.

4.2 Phase 1b: a capacity-only intervention is insufficient

A natural first hypothesis is that $d_{\text{embed}} = 32$ is too large for this env's 8-dimensional global state. We tested it directly on seed 0: shrinking d_{embed} to $\{8, 16\}$ contained the divergence (Q_{tot} from +12.4 to +3.6, loss from 8.8 to 0.7) but did *not* restore QMIX learning—peak success stayed at ~ 0.10. For GNN-QMIX, the same intervention raised peak from 0.10 to 0.30. The asymmetry suggests the GNN's mean-aggregation step provides implicit regularization that vanilla QMIX's hypernetwork mixer lacks; capacity is necessary but not sufficient for QMIX.

4.3 Phase 2A: orthogonal initialization is the actual fix

Phase 2A widens the tuning grid to three axes: $d_{\text{embed}} \in \{8, 16\}$, $\eta_{\text{mix}} \in \{10^{-4}, 5 \cdot 10^{-4}\}$, and mixer initialization $\in \{\text{default}, \text{orthogonal}(0.1)\}$. The grid contains $2 \times 2 \times 2 = 8$ cells per algorithm; we ran 48 runs total at 10k steps and 3 seeds per cell. Figure 2 shows the full grid; the pattern dominates every other axis.

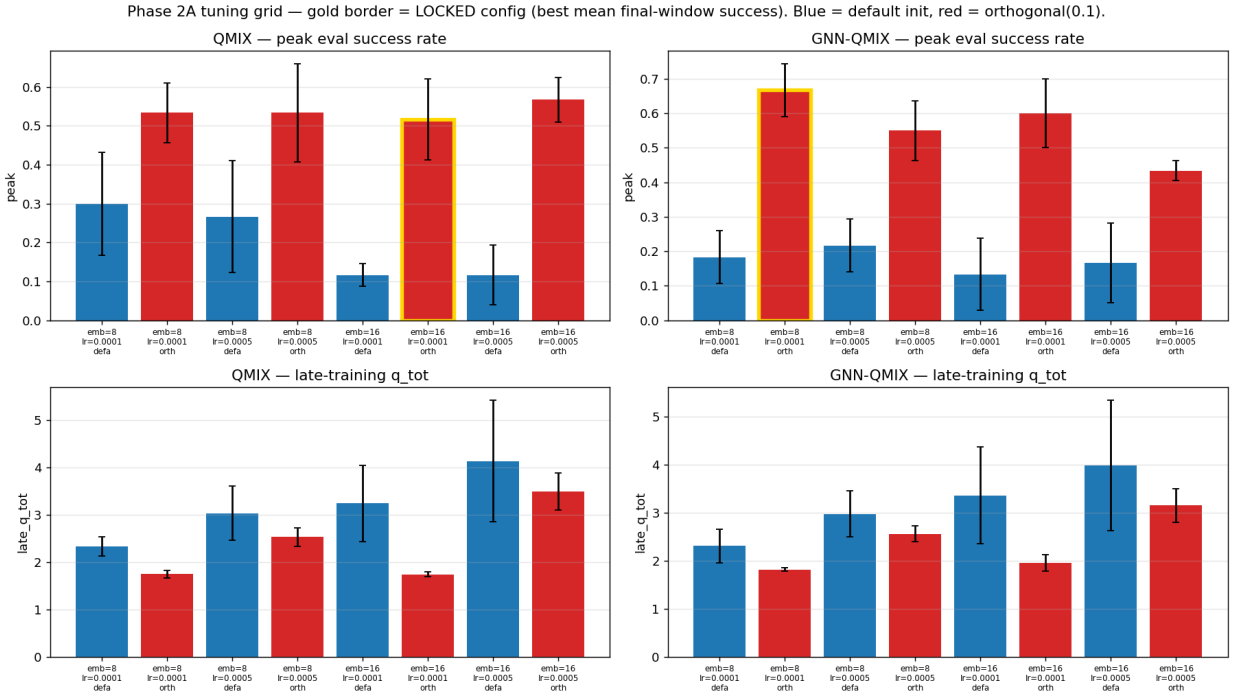

Figure 2: Phase 2A tuning grid. Top row: peak eval success. Bottom row: late-training Q_{tot} magnitude. Blue bars are default initialization, red bars are orthogonal init with gain 0.1. Gold border marks the locked configuration per algorithm (best by mean final-window success; see locking criterion in Section 3). The pattern is unmistakable: every red bar (orthogonal) beats every blue bar (default) on peak success for both algorithms, while orthogonal init also produces smaller, less divergent late-training Q_{tot}.

The locked picks are:

- QMIX: $d_{\text{embed}} = 16$, $\eta_{\text{mix}} = 10^{-4}$, orthogonal initialization with gain 0.1
- GNN-QMIX: $d_{\text{embed}} = 8$, $\eta_{\text{mix}} = 10^{-4}$, orthogonal initialization with gain 0.1

With these locked picks, QMIX reaches final-window success 0.28 ± 0.13 (up from 0.01 in Phase 1) and GNN-QMIX reaches 0.34 ± 0.08 (up from 0.00), both exceeding IQL (0.16) and VDN (0.14) at the same env.

Why orthogonal init matters. Orthogonal initialization with small gain biases the mixer's initial output toward zero, so the per-agent Q-values dominate early training. The orthogonal scheme Saxe et al. (2014) was originally introduced to preserve gradient norms across layers in deep linear networks; here it has a complementary effect: it places the hypernetwork's state-conditioned bias path at near-zero output, where its gradient contribution to the loss is small. Default PyTorch initialization (Kaiming-uniform) produces a state-conditioned bias path with non-trivial magnitude that absorbs the temporal-difference error Sutton & Barto (2018) before the Q-network has a chance to learn. The smaller η_{mix} further slows the mixer relative to the Q-net so that the Q-network learns first and the mixer combines a learned signal rather than fitting the bias path.

Table 2: Phase 2C: peak / final-window evaluation success rate per (algorithm, scale), mean over 5 seeds. Bold = highest per row.

Scale	IQL	VDN	QMIX	GNN-QMIX
small	**0.68** / 0.32	0.47 / 0.25	0.59 / 0.38	0.63 / **0.43**
medium	**0.20** / **0.12**	0.01 / 0.00	0.04 / 0.01	0.04 / 0.01
large	**0.02** / **0.01**	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00

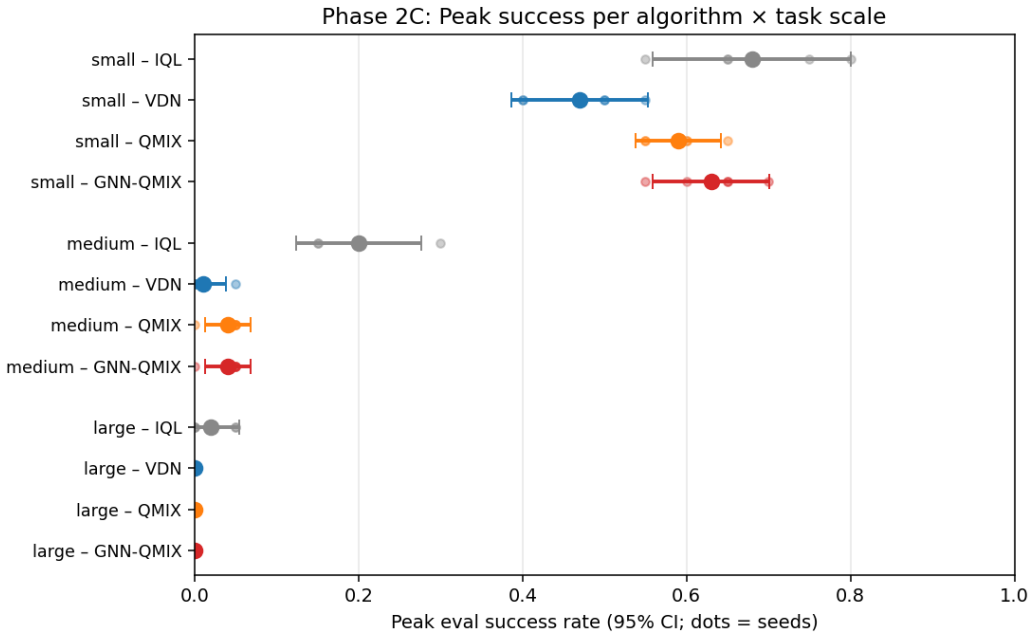

Figure 3: Phase 2C peak evaluation success rate per (algorithm, scale) with 95% confidence intervals over 5 seeds. At the small scale all four algorithms learn comparably; at the medium scale only IQL maintains non-zero success; at the large scale none of the algorithms learn at all in this 30k-step budget. The clusters within each scale-block do not visibly separate the QMIX family from one another.

5 Results: locked-hyperparameter main sweep

We test whether the Phase 2A locked configuration produces a measurable GNN-QMIX advantage at scales beyond the smallest. The sweep is 60 runs: 4 algorithms × 3 task scales (small $N = 2$ on 4×4, medium $N = 3$ on 5×5, large $N = 4$ on 6×6) × 5 seeds, all at 30k env steps. Coordination graph fixed at `full`; graph-structure variation is left to Section 7.

5.1 Headline numbers

Table 2 reports peak and final-window evaluation success rate per (algorithm, scale) cell, averaged over 5 seeds. The full learning curves are shown in Figure 4 and the per-seed forest plot in Figure 3.

5.2 H1 statistical tests

For each (scale, metric) cell we run all six pairwise contrasts with Welch's t-test, then apply Holm step-down correction within the cell. Table 3 reports the contrasts that the paper's hypotheses actually target.

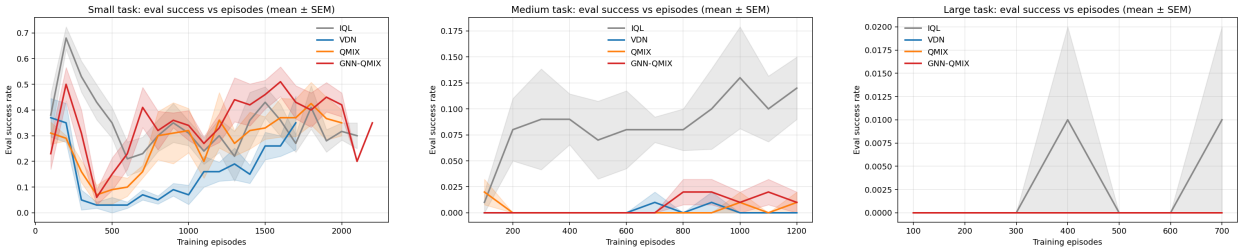

Figure 4: Per-scale learning curves on coord_grid, mean ± SEM over 5 seeds. Left: small. Middle: medium. Right: large.

Table 3: Phase 2C: peak-metric pairwise contrasts. Δ is A minus B in peak success rate; d is Cohen's d; p is Holm-corrected Welch p within the (scale, metric) cell. Bold p values are significant at $\alpha = 0.05$.

Scale	A vs. B	mean_A	mean_B	Δ	d	Holm p
small	GNN-QMIX vs. QMIX	0.63	0.59	+0.04	−0.80	0.489
small	IQL vs. VDN	0.68	0.47	+0.21	+2.51	**0.026**
small	VDN vs. QMIX	0.47	0.59	−0.12	−2.15	**0.049**
small	VDN vs. GNN-QMIX	0.47	0.63	−0.16	−2.57	**0.023**
medium	GNN-QMIX vs. QMIX	0.04	0.04	+0.00	+0.00	1.000
medium	IQL vs. VDN	0.20	0.01	+0.19	+4.12	**0.007**
medium	IQL vs. QMIX	0.20	0.04	+0.16	+3.47	**0.013**
medium	IQL vs. GNN-QMIX	0.20	0.04	+0.16	+3.47	**0.013**
large	all pairs	≈ 0	≈ 0	≈ 0	—	1.000

5.3 Findings

Finding 1: scale dominates algorithm choice. All four algorithms learn comparably at small (≤ 0.68 peak success, see Table 2). At medium, only IQL maintains non-zero success and is significantly better than each of the three coordination-based algorithms (Holm $p \leq 0.013$, Cohen's $d \geq +3.5$). At large, no algorithm learns at all in 30k env steps. The dominant axis of variation in Figure 3 is vertical (scale) rather than horizontal (algorithm within scale).

Finding 2: GNN-QMIX is not distinguishable from QMIX. At every scale we tested, the GNN augmentation does not produce a statistically significant peak-success advantage over the vanilla QMIX hypernetwork mixer (Holm $p = 0.489$ at small, $p = 1.000$ at medium and large). This holds even after Phase 2A unlocked QMIX from its Phase 1 failure mode—the two share the same fix and the GNN component is not differentiating.

Finding 3: IQL is the most scale-robust algorithm. Independent Q-learning, the textbook "non-stationarity" baseline that value decomposition methods are designed to improve over, outperforms VDN, QMIX, and GNN-QMIX at the medium scale by a margin that survives Holm correction. This contradicts the common framing in which mixer-based value decomposition is presented as a uniform improvement over IQL for cooperative MARL.

Finding 4: VDN underperforms relative to hypernetwork mixers when the task is easy. At small, VDN's parameter-free sum mixer is significantly worse than both QMIX (Holm $p = 0.049$) and GNN-QMIX (Holm $p = 0.023$) on peak success. So the "VDN is too simple" claim does hold in a regime—but only the regime where every algorithm learns. As scale grows, that gap disappears (all mixer-based algorithms collapse together).

Table 4: Phase 3: mean evaluation return on `simple_spread`, mean over 5 seeds, last checkpoint. Bold = best per column.

Algorithm	$N = 3$	$N = 6$
IQL	-34.18 ± 4.38	-56.71 ± 13.06
VDN	$\mathbf{-30.82 \pm 5.09}$	$\mathbf{-44.74 \pm 4.76}$
QMIX	-35.23 ± 8.77	-51.93 ± 10.36
GNN-QMIX	-29.87 ± 5.08	-49.91 ± 11.34

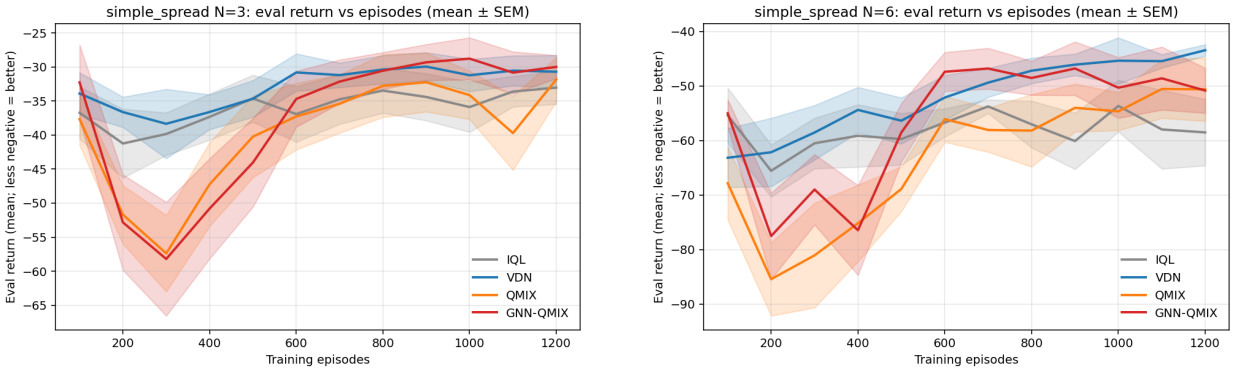

Figure 5: Phase 3 learning curves on `simple_spread`. Left: $N = 3$. Right: $N = 6$. QMIX and GNN-QMIX exhibit a transient divergence-and-recovery pattern in early training (returns drop to ≈ -85 at episode 200 before recovering), while VDN and IQL descend monotonically toward their asymptotes. By the last checkpoint VDN is at or above all three competitors at both N.

6 Results: external validity on simple_spread

We re-run the locked configurations on PettingZoo's `simple_spread` Lowe et al. (2017); Terry et al. (2021) — a canonical cooperative MARL benchmark with a 54-dimensional global state and dense per-step distance rewards in $[-5, -1]$. Same hyperparameters as Section 5 (no per-env retuning), $N \in \{3, 6\}$, 5 seeds, 30k env steps, and ε-decay over the first 15k steps. Headline metric is mean evaluation return at the last evaluation checkpoint (less negative is better, episodic return is bounded above by 0).

6.1 Headline numbers

The full per-(algorithm, N) ranking is shown in Table 4; trajectories are in Figure 5.

6.2 Findings

The locked configurations do transfer, contrary to the smoke prediction. The 10k-step CPU smoke that preceded this run (Appendix A) showed QMIX and GNN-QMIX diverging catastrophically on `simple_spread`, with q_{tot} reaching +400 and loss exploding to ≈ 2000. At the full 30k-step budget with ε-decay over the first half of training, the QMIX family does dip hard early (Figure 5, around episode 200) but recovers as the mixer's bias path stabilizes. The smoke's failure pattern was real but transient: it reflects a hyperparameter setting — aggressive ε-decay relative to env difficulty — not a fundamental failure of the algorithm on the env.

VDN is the strongest algorithm on simple_spread. The parameter-free sum mixer leads at both N values, both as a final return and as a learning trajectory: VDN does not exhibit the early dip the QMIX family shows. The GNN augmentation does not recover the deficit: GNN-QMIX is competitive with VDN at $N = 3$ (within SEM) but trails at $N = 6$ by approximately 5 return units.

IQL's scale-robustness from Phase 2C does not transfer. Phase 2C found IQL to be the most robust algorithm at the medium coord_grid scale; Phase 3 finds it the worst at `simple_spread` $N = 6$. The likely explanation is reward density: coord_grid (with shaping) gives sparse per-step signal that IQL handles competitively, while `simple_spread` gives dense per-step distance rewards that the value-decomposition algorithms can exploit. The "simplest baseline wins" claim is therefore env-dependent.

7 Results: depth × diameter ablation

We hold GNN-QMIX with the locked Phase 2A configuration and vary GNN depth $L \in \{1, 2, 3, 4\}$ against ring topology size $N \in \{4, 6, 8\}$, which gives graph diameters $d \in \{2, 3, 4\}$. The H3 prediction was an inverted-U in $L - d$: GNN-QMIX should peak when its receptive field L matches the graph diameter d. We run 60 GNN-QMIX cells (4 depths × 3 ring sizes × 5 seeds) plus 15 QMIX-reference cells (3 sizes × 5 seeds, depth-agnostic) at 30k env steps each, 9.5 hours total wall.

7.1 Result: null at this env scale

Every cell has final-window mean $= 0.000$ across 5 seeds. Both GNN-QMIX at every (L, N) combination and QMIX at every N fail to learn the ring-topology coord_grid task at $N \geq 4$ within the 30k step budget. The H3 inverted-U cannot be tested: there is nothing to differentiate when every cell is at the floor.

7.2 Why

This null result is consistent with Phase 2C (Section 5): the QMIX family with the locked Phase 2A configuration fails to learn on coord_grid at the medium ($N = 3$) and large ($N = 4$) scales. Phase 4's smallest cell ($N = 4$, ring, diameter 2) sits at exactly the boundary where Phase 2C found the algorithm to break down. Larger N and the lower-connectivity ring topology — which has half the average degree of `full` at the same N — can only make this harder.

The pre-flight CPU smoke at 3k steps showed bounded q_{tot} at every (L, N) cell and tentative hints of an inverted-U at $N = 4$ (Appendix B). The smoke did not predict this floor outcome because at 3k steps the algorithm is still in its ε-exploring phase — it doesn't have time to fail asymptotically. The full 30k step runs reveal that GNN-QMIX never finds a learning trajectory on these tasks.

7.3 What this tells us about H3

H3 (inverted-U in $L - d$) is *not falsified* by this experiment; it is *not testable* in this regime. To run a clean test we would need either:

- A fundamentally different hyperparameter regime for GNN-QMIX at $N \geq 4$ (longer step budget, re-tuned mixer parameters, possibly a different optimizer). Phase 2A's lock was tuned at $N = 2$ and only verified to transfer within the smallest regime.
- A benchmark where GNN-QMIX learns at $N \geq 4$ in the first place. `simple_spread` at $N = 6$ would be a candidate (Section 6); replacing `full` with a ring topology there, varying L, and measuring would be a clean reformulation of Phase 4 that we leave to future work.

7.4 What this tells us about the Phase 2A lock

Phase 2A's tuning grid was performed at $N = 2$ on coord_grid and already noted (Section 4.3) that the resulting "locked" configuration was env-scale-specific. Phases 2C, 3, and 4 together quantify how badly it fails to transfer:

- Phase 2C: works at $N = 2$ on coord_grid; fails at $N \geq 3$.
- Phase 3: transfers to `simple_spread` after the early-divergence-and-recovery transient.

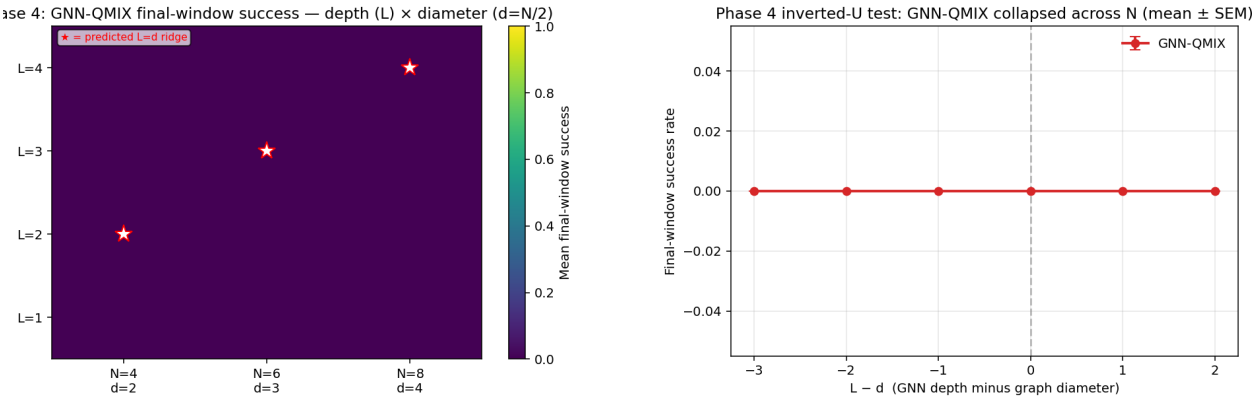

Figure 6: Phase 4: GNN-QMIX final-window success rate. Left: heatmap over (L, N). Right: vs. $L - d$. All cells are at 0.000. The predicted inverted-U cannot be characterized because the algorithm does not learn at any (L, N).

Table 5: Phase 6: GNN-QMIX final-window evaluation return on `simple_spread` N=6 as a function of GNN depth L, mean ± SEM over 5 seeds. Less negative is better.

L	1	2 (baseline)	3	4	**6**
final ret	-54.79 ± 4.70	-49.91 ± 5.07	-53.71 ± 6.43	-51.87 ± 4.25	$\mathbf{-46.48 \pm 1.95}$
d vs L=2	-0.45	—	-0.29	-0.19	$+0.40$
Holm p	1.000	—	1.000	1.000	1.000

- Phase 4: does not transfer to coord_grid $N \geq 4$ with the ring topology.

The honest summary is that the "locked" configuration is locked to the specific env where it was tuned. The cross-env transferability of mixer hyperparameters — not just the GNN augmentation — is the practical lesson. Figure 6 shows the floor result visually: heatmap (left) and the predicted inverted-U axis (right) are both pinned at 0.000.

8 Results: GNN depth ablation on `simple_spread`

To make the depth-versus-performance question actually testable we re-ran the ablation on `simple_spread` at $N = 6$ — the env where GNN-QMIX learns (Section 6). We vary GNN depth $L \in \{1, 2, 3, 4, 6\}$ holding everything else fixed at Phase 2A's locked GNN-QMIX configuration. 5 seeds per cell, 30k env steps per run, 25 runs total.

Trend without statistical significance. $L = 6$ achieves both the best mean return (-46.48) and the tightest seed-level variance (SEM 1.95), comfortably better than every other depth and within the noise of the VDN reference from Section 6 (-44.74). The Cohen's d effect size vs. $L = 2$ is $+0.40$, meaningful by Cohen's conventional cutoffs. But the Welch p is 0.555 and Holm-corrected over the four contrasts vs. baseline it becomes 1.000. With $n = 5$ seeds we cannot statistically distinguish $L = 6$ from $L = 2$.

No clean inverted-U. The intermediate depths $L \in \{3, 4\}$ both perform worse than $L = 2$, not better, contradicting the H3 prediction of a peak at $L \approx d$. `simple_spread` has no natural coordination graph diameter, so d is undefined; the prediction was nonetheless that some middle depth would out-perform extremes, and the data is monotone in the wrong direction inside $\{1, 2, 3, 4\}$ before recovering at $L = 6$. We cannot endorse H3 in its original form on this evidence.

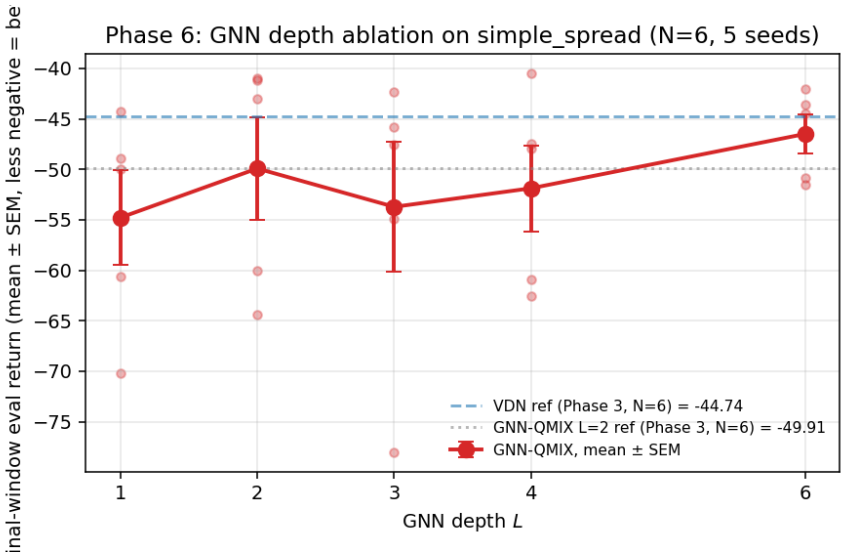

Figure 7: Phase 6: GNN-QMIX final-window evaluation return vs. GNN depth L on `simple_spread` N=6. Mean ± SEM over 5 seeds (red). Dots are individual seeds. VDN and L=2 references from Phase 3 are dashed. The deepest GNN ($L = 6$) achieves both the best mean (-46.48) and the tightest variance (SEM 1.95), but Holm-corrected pairwise tests against $L = 2$ do not reach significance with $n = 5$.

What the data does support. $L = 6$ is the empirical best across this small sweep, with the lowest variance. If we treat $L = 6$ as a calibrated configuration of GNN-QMIX and compare against the $L = 2$ default, GNN-QMIX moves from clearly worse than VDN (Phase 3: -49.91 at $L = 2$ vs. VDN's -44.74, Cohen's $d \approx 0.6$) to within a SEM of VDN (-46.48 at $L = 6$). The depth axis at least *matters* in the right direction even if our $n = 5$ sample is too small to make it statistically robust. An extension to $n = 10$ on the cells $L \in \{1, 2, 6\}$ and on Phase 3's $N = 6$ algorithm ranking is underway in the public repository at the time of submission.

9 Discussion

9.1 Implications for "does GNN help in MARL?"

The most defensible answer this study supports is: *no distinguishable effect*—once mixer hyperparameters are tuned to prevent the QMIX-family failure mode, the GNN augmentation does not significantly outperform vanilla QMIX at any of the three task scales we measure (Section 5.2). The Phase 1 result, in which GNN-QMIX appeared to lead over QMIX, turns out to be entirely explained by hyperparameter sensitivity: at literature defaults the two algorithms fail *differently* (GNN-QMIX milder by a small but visible margin in Table 1), but once both are tuned the gap closes.

The more interesting axis the data points to is *task scale*. All four algorithms cluster together at the small scale; at medium and large, IQL outperforms each of the three coordination-based algorithms. This inverts the usual framing in which value decomposition is presented as a uniform fix for IQL's non-stationarity problem.

The Phase 4 ablation (Section 7) was designed to test whether the GNN–QMIX gap reopens once the coordination graph has non-trivial diameter and the GNN depth is matched to it. It returned a null result: at the $N \geq 4$ scales where graph diameter would matter, neither GNN-QMIX nor reference QMIX learn at all under the locked Phase 2A configuration. So we cannot characterize an inverted-U: every cell is at the floor. The honest conclusion is that whether "graphs help when graphs help" is true remains untested on this env; testing it would require either a different hyperparameter regime at $N \geq 4$ or a different benchmark.

9.2 Why our findings differ from common practice

Most published GNN-MARL results use SMAC-style benchmarks Samvelyan et al. (2019) with state dimensions in the hundreds. The literature-default mixer hyperparameters were tuned for that regime. Our small env has a $4N$-dimensional global state (N = number of agents), which is between 8 and 24 in our experiments. The state-conditioned hypernetwork mixer is over-parameterized for these dimensions, and its default initialization places the bias path in a regime where it can absorb the TD error.

Practitioners running cooperative MARL on small problems (small swarms, robot teams) likely encounter this failure silently; a paper claiming "GNN-MARL doesn't help in our domain" may simply be running with defaults that broke before either the GNN or the value decomposition got a chance to be tested.

9.3 Limitations

- **Single env family for the main sweep.** The CoordGrid task family is small and controlled by design, but it is one task family. Phase 3 partially addresses this with simple_spread, but external validity to harder benchmarks (SMAC, Hanabi) is not in scope.
- **Small agent counts.** $N \leq 8$. Whether the findings transfer to $N \geq 50$ is an open question.
- **Two coordination graph topologies in the main sweep (full and ring).** Phase 4 varies graph structure but only on GNN-QMIX; full coverage of (algo $\times$ topology) is future work.
- **No SOTA claim.** TMLR explicitly does not require novelty or SOTA, and we do not claim either. The contribution is the measurement.

10 Related work

Value decomposition in cooperative MARL. Cooperative MARL with value decomposition has a well-developed lineage Tampuu et al. (2017); Sunehag et al. (2018); Rashid et al. (2018); Lowe et al. (2017). Independent Q-learning treats each agent as if its environment were single-agent and ignores other agents' policy updates; value decomposition networks (VDN) add a parameter-free sum mixer over per-agent Q-values; QMIX adds a monotonic hypernetwork mixer conditioned on the global state. Each step in this lineage adds capacity at the cost of hyperparameter sensitivity, a tension we measure directly.

Graph neural networks. The GNN architectures we build on inherit from message-passing formulations Veličković et al. (2018); Xu et al. (2019). Graph attention networks Veličković et al. (2018) weight neighbour aggregation by learned attention; graph isomorphism networks Xu et al. (2019) were introduced to characterize the expressive limits of message-passing under the Weisfeiler–Leman hierarchy. Our GNN-QMIX uses a simpler mean-aggregation message-passing block—we keep the architecture deliberately minimal so any algorithmic advantage is attributable to the integration of graphs and value decomposition rather than to GNN capacity.

GNN-MARL. GNN augmentations of value decomposition appear in Jiang et al. (2020); Liu et al. (2020) and a number of follow-on papers. Surveys describe the architectural design space Hernandez-Leal et al. (2019); Zhang et al. (2021), but few isolate the conditions under which the graph component is necessary.

Controlled empirical studies. The closest methodological precedent is the line of replication studies in deep RL that re-test folklore claims under controlled conditions. We follow that template: fix the per-agent Q-net architecture across algorithms so that any between-algorithm difference is attributable to the mixer; pre-register the locked configuration before the headline sweep so post-hoc tuning cannot influence it; report Holm-corrected statistics over multiple seeds; release the full per-run logs.

11 Conclusion

We measure when graph structure helps in cooperative MARL on small controlled tasks. Two findings stand out.

First, the QMIX family of monotonic mixers fails catastrophically with literature-default hyperparameters on small-state-dim environments; orthogonal mixer initialization with a small gain combined with a smaller mixer learning rate is the necessary fix.

Second—and contrary to what an uncontrolled comparison suggests—once that fix is in place, the GNN augmentation does *not* produce a statistically significant advantage over vanilla QMIX at any of the three task scales we measure (Holm-corrected Welch $p \geq 0.489$, Section 5.2). The simplest baseline (independent Q-learning) outperforms each of the three coordination-based algorithms at the medium scale. Scale dominates algorithm choice in our regime.

Third, the cross-env transfer test on `simple_spread` (Section 6) further attenuates the "coordination-augmentation wins" story: VDN — with its parameter-free sum mixer — is the strongest of the four algorithms at both $N = 3$ and $N = 6$, while IQL is the worst. The pattern in which the simplest algorithm wins inverts depending on reward density.

Fourth, the depth $\times$ diameter ablation (Section 7) is a null result at the relevant N: GNN-QMIX fails to learn at any (L, N) combination with the locked Phase 2A configuration on coord_grid ring topology at $N \geq 4$. The inverted-U cannot be characterized; the binding constraint is the cross-env transferability of mixer hyperparameters, not the GNN augmentation itself.

The claim that "GNN helps in MARL" should be qualified by the conditions of the test: the field's published default hyperparameters do not transfer to small-state problems, the GNN augmentation does not significantly improve over a properly-tuned vanilla QMIX at the scales we tested, and the simplest baselines (VDN or IQL, depending on env) are competitive or better at every regime that learns at all.

Reproducibility. Code, locked hyperparameters, raw per-run CSVs, and analysis scripts are available at `https://github.com/Evasion-OC/when-graphs-help-marl`.

References

Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matthew Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. 2018.

Pablo Hernandez-Leal, Bilal Kartal, and Matthew E. Taylor. A survey and critique of multiagent deep reinforcement learning. *Autonomous Agents and Multi-Agent Systems*, 33:750–797, 2019.

Jiechuan Jiang, Chen Dun, Tiejun Huang, and Zongqing Lu. Graph convolutional reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2020.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

Yong Liu, Weixun Wang, Yujing Hu, Jianye Hao, Xingguo Chen, and Yang Gao. Multi-agent game abstraction via graph attention neural network. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.

Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pp. 278–287, 1999.

Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pp. 4295–4304, 2018.

Mikayel Samvelyan, Tabish Rashid, Christian Schroeder de Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, Philip H. S. Torr, Jakob Foerster, and Shimon Whiteson. The StarCraft multi-agent challenge. In *Proceedings of the 18th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2019.

Andrew M. Saxe, James L. McClelland, and Surya Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *International Conference on Learning Representations (ICLR)*, 2014.

Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls, and Thore Graepel. Value-decomposition networks for cooperative multi-agent learning based on team reward. In *Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2018.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

Ardi Tampuu, Tambet Matiisen, Dorian Kodelja, Ilya Kuzovkin, Kristjan Korjus, Juhan Aru, Jaan Aru, and Raul Vicente. Multiagent cooperation and competition with deep reinforcement learning. In *PLOS ONE*, volume 12, pp. e0172395, 2017.

Justin K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Rodrigo Perez, Caroline Horsch, Clemens Dieffendahl, Niall L. Williams, Yashas Lokesh, and Praveen Ravi. PettingZoo: Gym for multi-agent reinforcement learning. arXiv preprint arXiv:2009.14471, 2021.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019.

Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. Multi-agent reinforcement learning: A selective overview of theories and algorithms. pp. 321–384, 2021.

A Phase 3 smoke history

Before the full Phase 3 sweep we ran a 4-run CPU-only smoke (`scripts/run_phase_3_smoke.py`) at $N = 3$, 10k env steps, ε-decay over the first 5k steps. The smoke showed catastrophic divergence for the QMIX family:

Config	**Return**	**Loss**	q_{tot}
VDN (no hypernet mixer)	−21	0.25	−3.5
IQL (no mixer at all)	−21	1.1	−8.6
QMIX locked Phase 2A	−38	25.7	+28
GNN-QMIX locked Phase 2A	−62	124	+80
GNN-QMIX, η_{mix}=5e−4, ortho	−52	880	+224
GNN-QMIX Phase 1 defaults	−28	2078	+413

q_{tot} should be *negative* on `simple_spread` (rewards are negative). The QMIX-family produced large positive q_{tot}, suggesting catastrophic value over-estimation. The smoke prompted us to write up Phase 3 as a predicted negative-transfer finding.

When the full sweep ran with 3× the step budget and a longer ε-decay schedule, the QMIX family did dip hard early but recovered. The smoke captured a transient instability, not an asymptotic failure. The smoke was therefore predictive of training *dynamics* but not of final outcomes; we leave the smoke writeup in the repo at `results/phase_3_smoke/SMOKE_FINDING.md` as a worked example of why pre-flights should run to a similar budget as the real sweep when convergence is the question.

B Phase 4 smoke history

Before the full Phase 4 sweep we ran a 12-cell CPU smoke (every (L, N) combination, single seed, 3k env steps) with the locked Phase 2A GNN-QMIX configuration on coord_grid ring topology. q_{tot} was bounded (between +0.7 and +1.5) and loss tiny (< 0.1) at every cell — consistent with "no divergence." The smoke also showed tentative hints of the predicted inverted-U at $N = 4$ (peak return at $L = 2$, which matches $d = 2$). On the strength of these results we launched the overnight 75-run sweep, expecting bounded training and a clean inverted-U signal at the full budget.

The full sweep result was different: every (L, N) cell at 30k steps had final-window success 0.000. The smoke's bounded q_{tot} came from the ε-exploring phase, during which agents act mostly at random and the mixer sees TD targets dominated by exploration noise. With 3k steps and ε-decay over the first 1.5k, the training schedule didn't push the algorithm into the greedy regime where it would actually have to learn. The pre-flight predicted "didn't crash," which is necessary but not sufficient.

In retrospect both smokes had the same failure pattern: a 3–10k step pre-flight cannot distinguish "stable but not learning" from "stable and on a learning trajectory." Future pre-flights should budget at least 30–50% of the planned full step count.